

AFP 项目完全指南

从零到专家的新人上手教程

写给 DGX 实验机的操作者

硬件: NVIDIA DGX Spark GB10 121 GB 统一内存 ARM64 + CUDA 13.0

2026 年 7 月 23 日

1 三十秒搞懂这个项目

我们拿一个 13 亿参数的语言模型 Pythia，分别喂给它代码推理数据和医学推理数据，对它做全参数微调。然后测量两件事：

1. 模型的权重被推了多远？（学名叫“权重散度”，记作 ΔW ）
2. 两个微调后的模型之间，能直接画一条直线走过去吗？（学名叫“线性模式连通性”，简称 LMC）

核心发现：决定连通性障碍高低的是训练稳定性，不是领域差异。医学数据上的训练稳定性比代码数据差 3 倍。

我们的论文标题是 *Training Stability, Not Domain Difference, Determines Linear Mode Connectivity*，翻译过来就是：**训练稳定性，而非领域差异，决定线性模式连通性**。当前是第 19 版，一共 20 页，覆盖 4 个领域、2 个模型家族，还带一章理论分析。

2 你的硬件：DGX Spark GB10

- **121 GB 统一内存** — CPU 和 GPU 共享同一块物理内存，没有独立的显存。
- **ARM64 CPU** — 这意味着 `torch.compile()` **不可用**（Triton 只支持 x86）。
- **CUDA 13.0**，Blackwell 架构（算力 12.1）。
- 训练速度: Pythia-1.4B 全参数微调, batch=128, seq_len=256: ≈ 5.5 秒/步 (≈ 42 分钟/epoch)。

绝对禁止：清除 CPU-GPU 共享内存。不要运行 `ipcrm`，不要手动清理 `/dev/shm`，不要做任何释放共享内存的操作。

后果：Tailscale 远程连接会永久丢失，只能物理接触机器才能恢复。DGX 的统一内存由 Tailscale 运行时状态和 GPU 计算共享，清除即断连。

3 项目地图：每个文件是干什么的

路径	类型	内容
CLAUDE.md	指令	AI 必须遵守的规则；核心数据表；“禁止做”清单
README.md	公开	GitHub 首页；论文标题和摘要级概述
paper.pdf	产出	编译好的 20 页论文（权威版本）
docs/internal/paper.tex	源码	LaTeX 源码 v19 — 论文的权威文本
docs/internal/PAPER.md	镜像	论文的 Markdown 版（LaTeX 才是权威）
docs/internal/APPENDIX.md	数据	附录数值表格
docs/internal/ENGINEERING.md	手册	23 条已知 Bug ，含根因 + 修复 + 教训
docs/internal/ICLR_SPRINT_PLAN.md	路线图	3 阶段实验计划、时间线、GPU 预算
docs/internal/DATA_INVENTORY.md	清单	全部 42+ 模型和 110+ 次扫描的完整目录
docs/internal/EXPERT_PANEL_FINDINGS.md	评审	6 专家对抗评审（v4 时期，历史文档）
docs/reports/FIGURE_SPEC_V2.md	规格	图表规格 — 精确数据、颜色、布局
docs/reports/figure_data.json	数据	Pythia 主要结果（图 1–3）
docs/reports/fig*.pdf	产出	论文插图（矢量 PDF）
docs/reports/ONBOARDING_GUIDE.pdf	教程	本文档
docs/reports/BUGFIX_TOKENIZE.md	报告	Tokenize 内存爆炸修复详情
docs/reports/INCIDENT_REPORT_TOKENIZE_OOM.md	报告	完整事故报告
scripts/train_agent.py	代码	全参数微调，含 drive-putt 学习率调度
scripts/lmc_barrier_scan.py	代码	11 点 LMC 障碍扫描
scripts/lmc_3pt_scan.py	代码	快速 3 点 LMC 扫描（OPT 用）
scripts/gptneo_pipeline.sh	流水线	GPT-Neo 全流程：tokenize + 训练 + LMC
scripts/generate_figures.py	可视化	从 figure_data.json 生成图 1–3
scripts/generate_fig4.py	可视化	生成图 4（跨架构复制）
experiments/trained_models/	数据	Pythia-1.4B 检查点（288 GB，240 个 .pt 文件）
experiments/trained_models_opt/	数据	OPT-1.3B 检查点（184 GB）
experiments/phase0_ivn/results/	数据	122 个 LMC 障碍 JSON — 核心数据
experiments/phase0_opt/results/	数据	OPT 复制实验结果
data/versaprm/	数据	已 tokenize 的训练数据（2.1 GB）

4 核心数据表

条件	ΔW	Code 障碍	Medical 障碍
标准散度	1.4%	0.053 ± 0.011	0.051 ± 0.013
高散度	8.0%	0.118 ± 0.031	0.228 ± 0.102
域内基线 (code)	—	0.048	—
域内基线 (medical)	—	—	0.147
噪声地板 (同模型)	—	≈ 0.000	≈ 0.000
噪声地板 (随机初始化)	—	0.033 ± 0.001	0.150 ± 0.019

表 1: 核心结果表。来源: CLAUDE.md 和论文 §4.2–4.5。

这些数字意味着什么

- 标准微调下 LMC 成立。** 障碍 ≈ 0.05 很小。从同一个预训练检查点出发微调的 code 模型和 medical 模型，仍然在同一个 loss 盆地中。你可以在这两个模型之间做线性插值，不会遇到大的 loss 惩罚。
- Medical 异常。** Medical 的域内障碍 (0.147) 是跨域障碍 (0.051) 的 **3 倍**。两个在同一份医学数据上训练的模型，它们在 loss 地貌中的差异，比一个 code 模型和一个 medical 模型还要大。医学数据的训练本质上更不稳定。
- Code 出奇稳定。** Code 域内 $\approx$ 跨域 ($0.048 \approx 0.053$)。对 code 来说，领域差异几乎不产生额外障碍。
- 障碍 $\neq$ 权重距离。** 对模型加 $\Delta W = 8\%$ 的高斯噪声，障碍只有 0.014。训练产生的 $\Delta W = 1.5\%$ 却产生障碍 0.053 — **9 倍差异**。决定障碍的是权重变化的方向，不是幅度。

5 所有实验一览

核心测量 (Pythia-1.4B, 1 epoch)

- 权重散度:** 对每对模型，逐层计算 $\Delta W_i = \|\theta_i - \theta_{\text{base}}\|_2 / \|\theta_{\text{base}}\|_2$ ，然后取平均。
- LMC 障碍扫描:** 11 点线性插值 $\theta(\alpha) = (1-\alpha)\theta_{\text{code}} + \alpha\theta_{\text{medical}}$, $\alpha \in \{0, 0.1, \dots, 1.0\}$ 。每个域用 2,000 条测试样本测 BCE loss。障碍定义: $\text{barrier} = \max_{\alpha} \mathcal{L}(\theta(\alpha)) - (\mathcal{L}(\theta(0)) + \mathcal{L}(\theta(1)))/2$ 。
- 域内基线:** 同域不同种子的模型对。障碍到底来自领域差异，还是种子方差？
- 种子对兼容性:** 高散度 medical 的 $C(5, 2) = 10$ 对全部测量。发现 **15 倍范围** (0.072–1.213) — 障碍是配对特定的，不是模型特定的。

机制实验

- 训练轨迹:** 每 40 步保存检查点。Code 呈现倒 U 形 (步 200 达峰，然后下降)。Medical 单调增长 (在 ≈ 0.21 趋于平稳)。
- 高斯扰动:** 加 5 个 ΔW 级别的各向同性噪声。即使 8% 散度也几乎没有障碍。证明障碍来自结构化的权重变化。

7. 分层选择性插值：只合并部分层。75% 的障碍集中在 0-7 层。深层 (16-23)：几乎为零。
8. 合并质量基准：纯权重平均优于 TIES-Merging 和 Task Arithmetic。Accuracy barrier 远小于 loss barrier。

跨架构复制 (OPT-1.3B)

OPT-1.3B (不同架构、不同训练语料、不同 tokenizer) 确认了 medical > code 的域内障碍模式 (比值: OPT 3.6× vs. Pythia 3.7×)。绝对障碍值 OPT 大 4-8 倍, 但相对排序保持一致。

GPT-Neo-1.3B

状态：待完成。训练脚本已就绪 (gptneo_pipeline.sh), 但结果尚未报告。

6 怎么跑实验

环境设置

```
export HF_ENDPOINT=https://hf-mirror.com
export HF_DATASETS_OFFLINE=1
export PYTHONUNBUFFERED=1
cd /home/jiayu/AFP
VENV=/home/jiayu/AFP/venv/bin/python3
```

训练一个模型 (≈42 分钟)

```
$VENV -u scripts/train_agent.py \
    --domain code --lr 1e-4 \
    --model-id EleutherAI/pythia-1.4b
```

输出到 experiments/trained_models/code/。

跑一次 LMC 障碍扫描 (≈15 分钟)

```
$VENV -u scripts/lmc_barrier_scan.py
```

批量流水线 (≈2-3 GPU-小时)

```
bash scripts/phase1_batch.sh      # 学习率扫描 + 域内基线
bash scripts/final_batch.sh       # 合并基准 + OPT 轨迹
bash scripts/theory_experiments.sh # 标签噪声 + 延长 putt + Hessian
```

快速状态检查

```
bash scripts/monitor.sh
```

7 十条铁律

这些规则每一条都曾经被违反过，每一次都造成了真实的损失。

1. **必须用 `nohup`** — 所有批量实验必须以 `nohup ... &` 启动。
2. **重定向输出** — `stdout/stderr` 写到 `experiments/` 下的时间戳日志文件。
3. **零 `stdin`** — 批量脚本里不能有 `input()` 或 `sys.stdin.read()`。
4. **`HF_DATASETS_OFFLINE=1`** — 训练/评估期间全程零网络。永远。
5. **禁用 `set -e`** — 一个实验挂了绝不能杀死整个批量。
6. **优雅失败** — 用 `|| { log "ERROR"; return 1; }`，绝不用 `exit 1`。
7. **输出存在即跳过** — `[ -f "$out" ] && continue`，保证幂等重跑。
8. **训练后必须验证 $\Delta W > 0.1\%$** — Bug 22 的教训：base 模型曾被当作“已训练”模型保存，重量变化为零，使所有早期 LMC 实验无效。
9. **启动后验证进程存活** — `sleep 3 && ps -p $PID`。
10. **写崩溃日志** — 结构化失败日志写到 `crashes/` 目录。

第 8 条是最重要的。Bug 22 浪费了数周的工作。每次训练后：计算 ΔW 对比 base 检查点。如果 $\Delta W < 0.1\%$ ，训练静默失败了，输出是垃圾。

8 绝对不能做的事

- 不要把 LR 写进结论或标题。LR 只是制造散度的旋钮，不是科学贡献。
- 不要提 AFP / IVN / gate。那个研究方向已废弃。VISION.md 仅为历史存档。
- 不要追“LMC 破裂”叙事。实验证明它没破。故事是关于稳定性不对称，不是失败。
- 不要用软链接管理模型目录。每个模型独立放在 `{domain}_lr{lr}_s{seed}/`。
- 不要盲目信任缓存。缓存文件名含 `MAX_LEN`。如果改了配置，必须删旧缓存。
- 不要在 tokenize 循环里用 `return_tensors='pt'`。每个样本创建一个 tensor 对象 — 大数据集直接 OOM。
- 不要把 token ID 存为 Python `list[int]`。Python int 比 tensor int64 大 3.5 倍。

9 23 条已知 Bug 中最关键的 5 条

完整列表见 docs/internal/ENGINEERING.md。

- 1. **Bug 11: 内层循环 .item() 10,416 次 GPU 同步。** MAS 重要性计算耗时 5+ 小时而非 1 分钟。修复: GPU 端累积, 最后一次性同步。
- 2. **Bug 12: MAS 在 CPU 上跑。** to_device() 调用在计算之后, 不在之前。修复: 在任何计算之前把模型移到 GPU。
- 3. **Bug 21: 训练后未验证 ΔW 。** Base 模型被当作“已训练”保存, 权重零变化。修复: 强制训练后 ΔW 检查。
- 4. **Bug 22: code_e1 实际上是没有训练过的 base 模型。** 所有早期 LMC 实验全部无效。“code”模型就是纯净的 Pythia-1.4B, 零微调。
- 5. **Bug 23: 多种子训练静默覆盖。** 输出路径没有区分种子。跑 3 个种子只产出了 1 个模型 (最后一个)。

10 论文结构

章节	内容
摘要	训练稳定性而非领域差异决定障碍高度
§1 引言	为什么这对模型合并和理解微调很重要
§2 相关工作	LMC、模型合并、微调分析 — 24 篇引用
§3 方法	Pythia-1.4B 全参数微调、11 点 LMC、drive-putt 学习率调 度
§4.1 权重散度	逐层 ΔW , 跨域 $r = 0.995$
§4.2 LMC 障碍	标准 vs. 高散度, 中间散度的灾难性尖峰
§4.3 域内基线	4 领域稳定性谱; 种子对兼容性 (C(5,2)=10 对)
§4.4 不对称性	Medical 微调正向迁移到 code, 反之不然
§4.5 噪声地板	同模型 ≈ 0 , 随机初始化参考 ≈ 0.150
§4.6 训练轨迹	Code 倒 U 形 vs. medical 单调增长
§4.7 高斯扰动	结构化位移的障碍是随机噪声的 $9\times$
§4.8 分层选择性插值	75% 在 0-7 层, 0% 在 16-23 层
§4.9 OPT 跨架构	跨架构复制: med/code 比值 OPT $3.6\times$ vs. Pythia $3.7\times$
§5 理论	Hessian 对齐的 SGD 扩散; 佩克莱数; 盆地统计
§6 讨论	合并质量、对持续学习的启示
§7 局限性	置换不变性已验证 (神经元相关 0.984)、单模型家族已处 理、收敛混淆

11 图表状态

图	面板	状态	内容
1	A-F (6)	已完成	LMC 曲线 (4) + 柱状图 + 逐层散度
2	A-D (4)	已完成	轨迹 (2) + 高斯校准 + 分层选择性
3	A-B (2)	已完成	种子对热力图 + 4 域稳定性谱
4	A-C (5)	已完成	Pythia vs. OPT: 障碍柱状图 + 比值 + 轨迹 + 高斯 + 逐层散度
5	A-D (4)	待 GPT-Neo	三模型跨架构对比
6	A-B (2)	待 GPT-Neo	三模型轨迹对比

12 内存预算：为什么 121 GB 会满

DGX 总共有 121 GB，CPU 和 GPU 共享。训练时的去向：

- 模型参数 (bf16): $1.4\text{B} \times 2 \text{ bytes} = 2.8 \text{ GB}$
- 梯度 (bf16): 2.8 GB
- AdamW 优化器状态 (fp32): $m + v = 2 \times 1.4\text{B} \times 4 = 11.2 \text{ GB}$
- 激活值 (batch=128, L=256): $\approx 4 \text{ GB}$
- 训练数据张量: $\approx 15 \text{ GB}$ (硬上限 30 GB)
- 开销 / CUDA 上下文: $\approx 5 \text{ GB}$

合计: $\approx 41 \text{ GB}$ (典型训练)。数据硬上限 (30 GB) 防止触及 121 GB 上限。

Tokenize 脚本刚修好 — 原始代码把 token ID 存为 Python list[int] (每个 int 28 bytes), 而不是 tensor int64 (每个 8 bytes)。1000 万个训练步骤仅此一项就浪费了 72 GB, 训练还没开始就 OOM 了。详见 docs/reports/BUGFIX_TOKENIZE.md 和 INCIDENT_REPORT_TOKENIZE_OOM.md。

13 当前差距 (2026 年 7 月)

Bug 待修 (0 GPU-小时)

1. lmc_barrier_scan.py 中的障碍公式计算的是 $\max(L) - L[0]$, 而不是正确的 Frankle 定义 $\max(L) - (L[0] + L[-1]) / 2$ 。论文数字是手动修正的。
2. lr=2e-4/3e-4 的 6 个 LMC 结果文件是字节完全相同的副本 (假数据)。模型确实存在, 但扫描从未跑过。
3. CLEANUP_MANIFEST.md 声称 4 个文件已删除, 但它们仍然在磁盘上 (VISION.md、DIRECTION.md、THEORY.md、OPT_SECOND_MODEL.md)。

实验待做

1. **GPT-Neo-1.3B 复制**（最高优先级）。状态：脚本存在，结果未知。
2. **OPT 升级到 11 点扫描**（目前轨迹/高斯仅用 3 点 LMC）。
3. **ICLR 冲刺计划 Phase 1**（ ≈ 9 GPU-小时）：重跑 $lr=2e-4/3e-4$ 扫描、解决上界矛盾、扩展高散度 medical 种子数。

14 新人提效指南

1. **先读 CLAUDE.md**。这是 AI 的指令文件 — 它告诉你项目是什么、不能做什么、以及十条铁律。
2. **读 ENGINEERING.md §2**。23 条 Bug 比任何教程都好 — 每条都教给你这个机器或代码库的一个非显而易见的特性。
3. **记住关键数据位置**：`experiments/phase0_ivn/results/`（122 个 JSON）、`paper.tex`（LaTeX 源码）、`FIGURE_SPEC_V2.md`（图表规格）。
4. **从一个小实验开始**。训练一个模型（`train_agent.py -domain code`），验证 ΔW ，跑一次 LMC 扫描。这条端到端路径验证一切是否正常。
5. **用 `scripts/monitor.sh` 检查正在跑什么和还有什么待做**。
6. **出问题时**，90% 的情况是已知 Bug。先查 `ENGINEERING.md`，再从头调试。
7. **做新实验时**，复制 `phase1_batch.sh` 或 `gptneo_pipeline.sh` 的模式。严格遵循十条铁律。

15 常用命令速查表

任务	命令
训练 code 模型	<code>\$VENV -u scripts/train_agent.py -domain code -lr 1e-4</code>
训练 medical 模型	<code>\$VENV -u scripts/train_agent.py -domain medical -lr 1e-4</code>
LMC 扫描 (11 点)	<code>\$VENV -u scripts/lmc_barrier_scan.py</code>
LMC 扫描 (3 点, 快速)	<code>\$VENV -u scripts/lmc_3pt_scan.py -model-a DIR_A -model-b DIR_B</code>
验证训练后 ΔW	<code>\$VENV -c "import torch; a=torch.load('base.pt'); b=torch.load('trained.pt'); ..."</code>
查看状态	<code>bash scripts/monitor.sh</code>
生成论文图	<code>\$VENV scripts/generate_figures.py && \$VENV scripts/generate_fig4.py</code>
编译论文	<code>cd docs/internal && pdflatex paper.tex && pdflatex paper.tex && cp paper.pdf ../../paper.pdf</code>
从 GitHub 同步	<code>git pull origin main</code>
GPT-Neo 全流程	<code>bash scripts/gptneo_pipeline.sh</code>

16 配色方案 (供画图参考)

实体	色号	颜色
Pythia / Code 领域	#2196F3	 蓝色
Medical 领域	#F44336	 红色
OPT	#FF5722	 深橙
GPT-Neo / General 领域	#4CAF50	 绿色
Math 领域	#FF9800	 橙色
高斯 / 噪声	#9E9E9E	 灰色
高散度	#9C27B0	 紫色

本指南用 L^AT_EX 编写。源码: docs/reports/ONBOARDING_GUIDE.tex。

用 xelatex ONBOARDING_GUIDE.tex 重新编译。

仓库: <https://github.com/FauReam/AFP>